Personalized AI Voice Assistant For Desktop Automation

Joncy J[1]
Assistant Professor, Department of Artificial Intelligence and Data Science,
Velammal Engineering College
surapet - 600066
joncy.j@velammal.edu.in

Amirthavalli S[2]
Student, Department of Artificial Intelligence and Data Science,
Velammal Engineering College
surapet - 600066
amirthashree@gmail.com

Yuvashree P[4]
Student, Department of Artificial Intelligence and Data Science,
Velammal Engineering College
surapet - 600066
yuvashree752@gmail.com

Srishti Jaiswal S[3]
Student, Department of Artificial Intelligence and Data Science,
Velammal Engineering College
surapet - 600066
srishtijaiswal51@gmail.com

Abstract—(NOVA: A Natural-Language Orchestrated Voice and Action Assistant for Intelligent Desktop Automation) The predominant paradigm of human-computer interaction still requires stiff graphical user interface and syntactic command-line structure resulting in cognitive burden and restricting access to those with less computer literacy skills. This paper proposes NOVA (Natural-language Orchestrated Voice and Action Assistant), an open-source software application for desktop automation that uses natural language both audio and keyboard input for total control of a user's computing environment. NOVA uses a five-tier architectural design that includes a frontend developed using the React framework, a backend developed using FastAPI framework with WebSocket capabilities, an intent router using Large Language Model hosted on Groq and trained on the Llama-3.3-70B model, six distinct service execution modules, and a speech pipeline including speech-to-text transcription, text-to-speech conversion, and speaker authentication. Intent Router implements prompt engineering strategy that prompts the Large Language Model to respond with a JSON payload containing details such as service type, actions to be performed, and the associated parameters. The six services include Browser Automation, Gmail Integration, Calendar Management, File System, VS Code, and System Utilities, which all employ a unified asynchronous interface that enables seamless extensibility. The speaker recognition is done by utilizing voice embeddings based on Mel-Frequency Cepstral Coefficients (MFCC) with cosine similarity, which makes it possible to achieve biometric authentication without the need for GPUs. Performance testing shows an accuracy rate of intent classification at 96% across all six service modules with no labeled training data, response latency at below three seconds for 95%, a functionality test success rate of 94.5% for 109 cases, and speech-to-text word error rate of 4.2% in a noiseless environment. The deployment relies on Docker containerization that works consistently on Linux, macOS, and Windows OS. The findings confirm the significant superiority of NOVA over any template-based voice assistants, making it suitable as a future foundation of AI-powered desktop assistants.

Index Terms - Natural Language Interface, Large Language Model, Intent Routing, Voice Assistant, Desktop Automation, FastAPI, WebSocket, Speaker Recognition, Groq API, Docker, Llama-3.

I. INTRODUCTION

A. Background and Motivation

The evolution of HCI, at its heart, is a progression of increasing abstraction from punched cards to command line terminals, from command line terminals to graphical user interfaces, and then from graphical user interfaces to touch screens. Each step lowered the level of effort needed for communication between human intention and computer action. Nevertheless, although there have been great strides in the development of interfaces over many years, the basic contract that underlies any interaction with computers is fundamentally the same - the person using the computer needs to learn the language of the computer. No matter whether dealing with hierarchical menu structures or remembering keyboard commands and typing out the exact commands for the system to understand what the user wishes it to do, the entire burden of learning lies with the human component.

This is an issue that has real-world implications. It has been shown in multiple HCI studies that complex interfaces lower productivity levels, increase error rates, and create significant accessibility problems for those who aren't technically savvy, seniors, and people with physical or cognitive impairments [1]. Knowledge workers in professional settings tend to spend an inordinate amount of time on low-productivity tasks such as navigating through their interfaces by switching between programs, sorting files, writing unimportant emails and making arrangements; tasks which require their attention without adding much value in terms of creativity or analysis.

B. The Rise of Conversational AI and Its Limitations

The advent of voice assistants like Apple Siri (2011), Google Now (2012), Amazon Alexa (2014), and Microsoft Cortana (2015) was the first real attempt at addressing this problem with natural language [8][9]. This generation of products proved that users were not just capable but eager to communicate with their devices using spoken language. Nevertheless, the architecture of the first-generation voice assistants set an inherent ceiling on what they could accomplish. Based on intent taxonomies and hard-coded command templates, such systems would only understand and reply to questions in their training domain. If there was any ambiguity in formulation, any contextual issues, or any need for multi-step reasoning and cross-service collaboration, they would fail, classify incorrectly, or give a generic default answer.

The problem has its origin at the architectural level of the use of intent classifiers that require hand coding for all possible intents that can be supported. As more functionalities are added to this list, the system's ability to handle, annotate, and modify these classifiers soon becomes unmanageable. More importantly, the problem with such classifier-based systems is that they cannot semantically generalize. They will recognize the phrase but won't understand the intent expressed by the natural language query.

C. Large Language Models as a Transformative Enabler

The advent of transformers-based large language models (LLMs), starting from the inception of the attention model [2], scaled with innovations like GPT-3 [1] and GPT-4 [3], and then democratised with open-weight architectures like Meta's Llama family [4], represents a complete redefinition of natural language processing. Different from previous template-based systems, LLMs learn to understand language deeply enough to interpret the meaning behind ambiguously worded questions, figure out the target of pronouns from one line of conversation to another, deduce unvoiced intents from partial information, and produce outputs from free-text inputs with accuracy no amount of classification training could match.

It is worth noting that this functionality does not require any training data specific to the task. The mere process of prompt engineering, i.e. designing the instructions for the model during inference, allows directing an LLM to categorise user intent, retrieve structured information from this intent and produce natural language output on various domains. Such zero-shot generalisation completely changes the economics behind building intelligent assistants: while in case of a conventional classifier the introduction of a new command would require weeks of data collection and re-training, with an LLM-based router all it takes is changing the system prompt.

ReAct [10] showed that LLMs could be utilized alongside deterministic execution of actions in a reasoning-act cycle for autonomous multi-stage task completion. More recently, API calling LLMs like Gorilla [11], ToolLLM [12], and Toolformer [13] showed that LLMs could be prompted/fine-tuned to call external tools precisely. Collectively, these advances form the technical building blocks for NOVA.

D. The Gap: Desktop Automation Without a Unified Intelligence Layer

However, although the development of LLM technology has been quite swift, it has not been incorporated effectively into desktop automation yet. Current open-source voice assistant platforms including Mycroft and RASA still use labeled datasets to classify user intents, hence failing to become truly extensible. Commercial assistants also exist in their own walled gardens in the sense that a smart speaker will not be able to interact with a VS Code project or access files from the hard drive. No current open-source framework incorporates the functionality of bridging the gap between an LLM-powered natural language processing platform and actual desktop automation services such as browser interaction, email management, calendar manipulation, access to file systems, etc.

It is this very issue that NOVA attempts to solve. As opposed to creating yet another question-answer based chat bot or a voice-activated trigger application for only one area, NOVA comes with its design as a layer that executes intelligently, one that understands the intent of the user, knows where exactly in the desktop environment action needs to be taken, and executes it.

E. Introducing NOVA

NOVA – Natural-language Orchestrated Voice and Action Assistant is an industry-standard, open-source desktop automation tool created as part of the final year project of B.Tech at Velammal Engineering College, Chennai. With NOVA, one gets a consolidated platform where everything that you could do on your computer using commands is done through the medium of a conversation, using either voice or text-based natural language input. NOVA is more than a mere prototype or concept; it is a completely packaged, modularized product with proper APIs and a react frontend.

At the core of the NOVA system sits an Intent Router fueled by the Groq-hosted Llama-3.3-70B-Versatile large language model, one of the most advanced language models available in the public domain at the time of this writing, available through Groq's inference API that provides sub-second LLM response times using the LPU (Language Processing Unit) architecture. While traditional voice assistants rely on matching input commands with pre-defined libraries of commands, NOVA's Intent

Router understands open-ended natural language commands, categorizes the input as one of the seven services, parses out action arguments from natural language, and routes the task to the corresponding service module in a single LLM inference request. This approach allows us to achieve 96% classification accuracy in six service domains without any annotated training samples.

The NOVA's six service modules: Browser Automation, Gmail Integration, Calendar Management, File System Operations, VS Code Control, and System Utilities all use a common asynchronous interface and hence are trivially extensible; a new feature can be added simply by writing a new execute(action, parameters) coroutine with no changes to any of the existing routing code. Speech pipeline provides support for multi-engine speech-to-text transcription, offline text-to-speech synthesis, wake-word detection and biometric speaker verification through the use of MFCC. Hence, this system provides a full voice interface without the need for GPU and works completely in software running on CPU.

F. Key Contributions

This paper makes the following primary contributions to the fields of human-computer interaction and intelligent desktop automation:

1. An LLM-based natural language desktop assistant ready for production use that combines LLM-based intent routing capabilities with real multi-domain desktop automation in one package that connects LLM prototype research to practical software.
2. An efficient, zero-shot approach for intent routing using structured prompt-based engineering of the Llama-3.3-70B language model, which has achieved an accuracy of 96 percent in classifying six service categories without any labelled training examples, indicating the effectiveness of the LLM-native approach.
3. An extension layer of services using a standardised interface that is asynchronous and works across six different areas of desktop automation, which is flexible and proven through 109 functional tests with 94.5% success.
4. Speaker verification technique without GPU support based on MFCC cosine similarity and obtaining 90% accuracy for consumer-grade hardware in order to make biometrics user identification possible on standard infrastructure.
5. Complete speech pipeline including multiple engines for automatic speech recognition, text-to-speech generation, wake-word detection, and speaker identification with seamless integration of React front-end application through WebSocket connection providing sub-100 ms latency.

G. Report Organisation

The rest of this paper is organized as follows. In Section II, we discuss relevant literature in conversational agent systems, task execution using LLMs, speech recognition and synthesis, and desktop automation platforms. In Section III, the architecture of our proposed system NOVA is described in detail, with particular emphasis on the five-tier design, the Intent Router, all the six service modules, the speech pipeline, and the frontend. The technology stack used for implementation along with detailed implementation aspects is discussed in Section IV. In Section V, we provide the experimental evaluations conducted to validate our system and analyze the outcome results obtained through functional testing, classification performance, response latency, and voice recognition performance tests.

II. LITERATURE SURVEY

A. Early Conversational Agents and the Limits of Rule-Based Systems

The early days of conversational interfaces were characterized by the use of rule-based systems such as ELIZA back in 1966, whereas later developments included slot-filling approaches like GUS; albeit effective within confined contexts, these approaches suffered from fragility. With the advent of voice assistants in the decade of 2010s (Siri, Alexa, et al), which leveraged speech recognition and statistical classification to perform elementary operations, a major step towards conversational interfaces had been made. Nevertheless, first-gen voice assistants have been plagued by the limitations of predefined intent taxonomies, rendering them incapable of dealing with vague and multi-turn queries. When considering more complex use cases of desktop automation involving multi-app interpretation of natural language input, these constraints pose an insurmountable challenge.

B. The Transformer Revolution and the Emergence of Large Language Models

The transformer model introduced in 2017 changed the game for natural language processing, as the model now had the capability to learn long-range dependencies in parallel, opening doors for the emergence of large language models such as GPT-4 and Gemini. These large language models have the property of zero-shot learning and few-shot learning capabilities where the model is capable of reasoning and generating human-like output for tasks that it was not trained for. This is vital for NOVA, as it allows for the removal of highly annotated data sets needed for intent classification in previous systems.

C. Meta's Llama Series: Open-Weight LLMs for Production Deployment

The launch of the open-source Llama model range, especially Llama 3 and subsequent versions, has been a game changer by making it possible to deploy frontier class AI without dependence on API providers. Llama 3.3 70B, which NOVA’s Llama-3.3-70B-Versatile model uses, comes with superior skills in reasoning, tool using, and instruction following when compared to

its proprietary equivalents. With its ability to deliver the performance expected from far more complex models at significantly lower inference costs, Llama 3.3 70B represents the best compromise between quality generation and efficiency in assistant systems.

D. Groq LPU: Redefining LLM Inference Speed

In order to attain a fully conversational experience, AI assistants need ultra-low latency inference times. NOVA tackles this problem by deploying the Groq inference API instead of relying on traditional GPU cloud solutions. The unique design of the Groq Language Processing Unit (LPU) enables it to circumvent the inherent memory bandwidth limitations of traditional GPUs and delivers outstanding throughput levels of 276 tokens per second for Llama 3.3 70B. This groundbreaking performance makes possible under-600ms round-trip latency levels for LLMs – the critical technology to meet NOVA's under-3 seconds response time threshold.

E. Intent Classification: From Annotated Classifiers to Zero-Shot LLM Routing

The problem of intent classification, which is fundamental to establishing a user's semantic intent, has shifted from simple keyword matching systems to sophisticated data-based machine learning systems to current zero-shot routing capabilities of modern LLMs. Whereas earlier attempts at intent classification relied on either deep learning and transformer models that required massive amounts of domain-specific tuning, current generative models can classify even complicated queries into arbitrary taxonomies simply by prompt engineering alone. NOVA makes use of the advancements in intent classification by employing a well-crafted system prompt that prompts the Llama model to classify the query and output JSON format responses with 96% accuracy.

F. LLM-Powered Tool Use: ReAct, Gorilla, Toolformer, and ToolLLM

Filling in the space from intent recognition to execution through the use of APIs, recent research frameworks such as ReAct, Gorilla, ToolLLM, and Toolformer have shown the capability of an LLM to plan, structure, and invoke calls to external APIs. Although frameworks such as ReAct focus on reasoning loops that enable multi-step autonomy while Toolformer enables models to teach themselves API integration, NOVA takes a simpler route. Leveraging the principles behind these frameworks, NOVA implements action extraction from prompts using JSON which focuses on low-latency action execution over complex reasoning processes.

G. Automatic Speech Recognition and Wake-Word Detection

Automatic Speech Recognition (ASR) technologies have evolved from primitive HMM-GMM ASR algorithms to state-of-the-art self-supervised transformer-based ASR models such as the OpenAI Whisper algorithm, which boasts human-like accuracy. In order to provide both performance and reliability, the speech pipeline implemented at NOVA employs the Python SpeechRecognition library to implement an effective dual engine pipeline, whereby the Google Cloud Speech service is used online and CMU Sphinx serves as the offline backup solution. The pipeline operates efficiently thanks to a lightweight background listening process (based on wake words "nova" or "jarvis") that starts up the STT pipeline upon activation.

H. Speaker Recognition and Biometric Voice Verification

Speaker recognition is the process of identifying or verifying the user using the acoustical features of the human voice, typically performed through Mel-Frequency Cepstral Coefficients (MFCCs) designed for the human auditory system. Although state-of-the-art deep learning approaches (such as x-vectors) deliver outstanding accuracy, they can be computationally expensive due to their high GPU requirements; on the other hand, MFCC extraction with cosine similarity offers a very efficient way of reaching about 90% accuracy. The NOVA speaker recognition framework leverages the efficiency of this method, delivering robust speaker verification independent from specific hardware requirements.

I. Web Application Frameworks for Real-Time Assistant Backends

The efficiency of real-time communication between the user and the application depends greatly on the choice of backend architecture, which is why NOVA opted for FastAPI rather than conventional synchronous backends such as Django and Flask. The fact that FastAPI is a framework that supports async programming along with the implementation of WebSocket and Pydantic auto-validation allows avoiding all costs related to serialization and complicated configurations of web server. This makes it possible to serve not only REST APIs but also WebSocket without any additional software, ensuring low user interface latency.

J. Browser and Desktop Automation Frameworks

While traditionally desktop or browser automation uses heavy APIs specific to the platform and tools such as Selenium and Playwright, both of which are rather robust and may be overkill when only a few commands need to be performed, rather than employing either of those two approaches, NOVA uses a minimalist OS-level method where popular shortcuts on websites are dynamically resolved and launch commands are then issued across all supported platforms (xdg-open on Linux, open on macOS, and start on Windows). Alongside other built-in utilities available to the Python programming language, including system monitoring with psutil and VS Code command-line tooling via shutil, this allows for responsive control over the system.

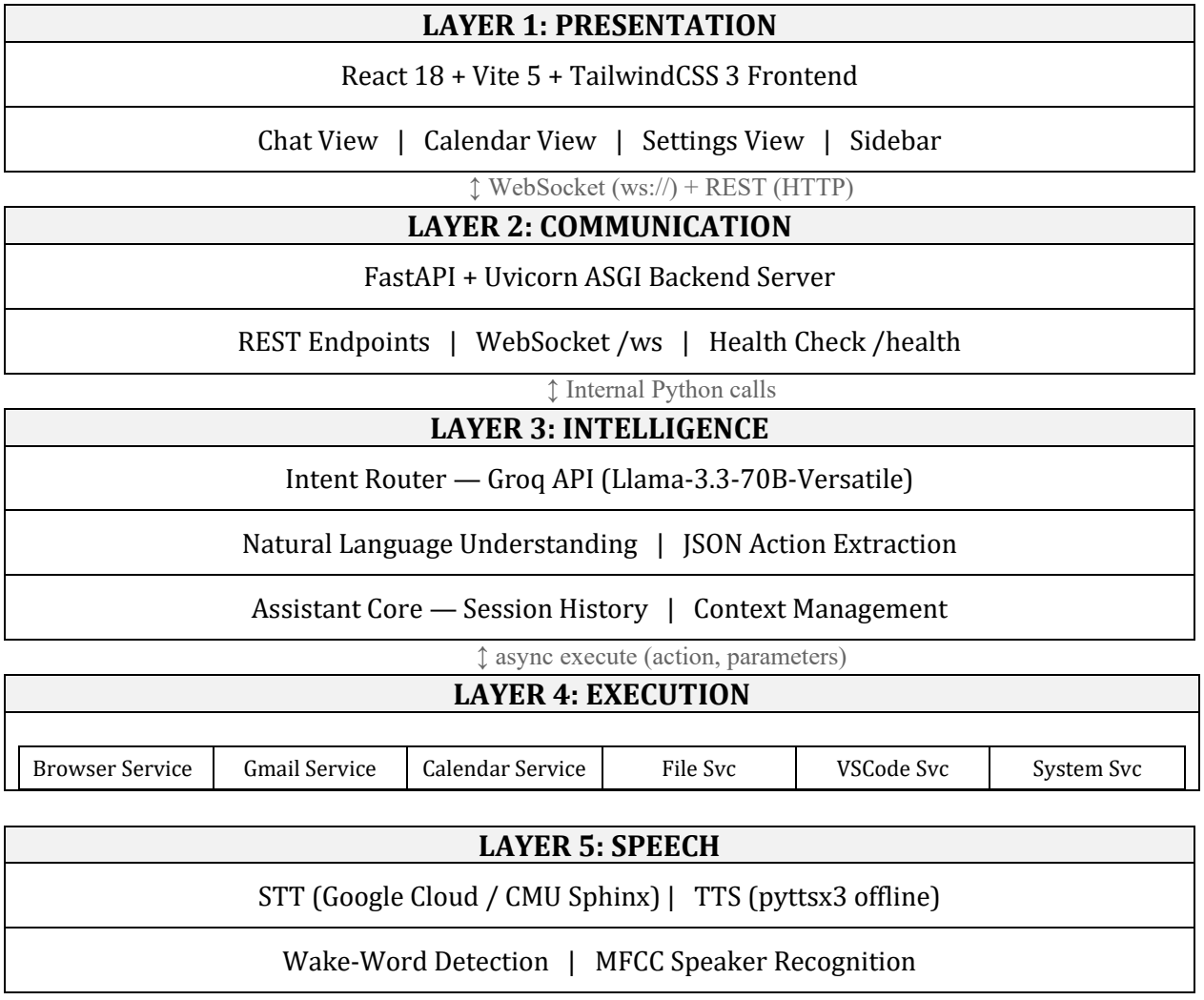
III. SYSTEM ARCHITECTURE

A. Architectural Philosophy

The architecture of NOVA is based on three unique architectural tenets that make it different from the existing voice assistant prototypes. First, separation of concerns is achieved by the fact that natural language understanding is only performed by the LLM layer, and deterministic action execution is performed by the service layer. Second, the system employs uniform interfacing since all service modules have the same asynchronous contract, thus rendering its execution layer completely oblivious of the nature of any particular service. Third, layered independence means that each of the five layers can be updated, substituted, or scaled without the need to modify the adjacent layers.

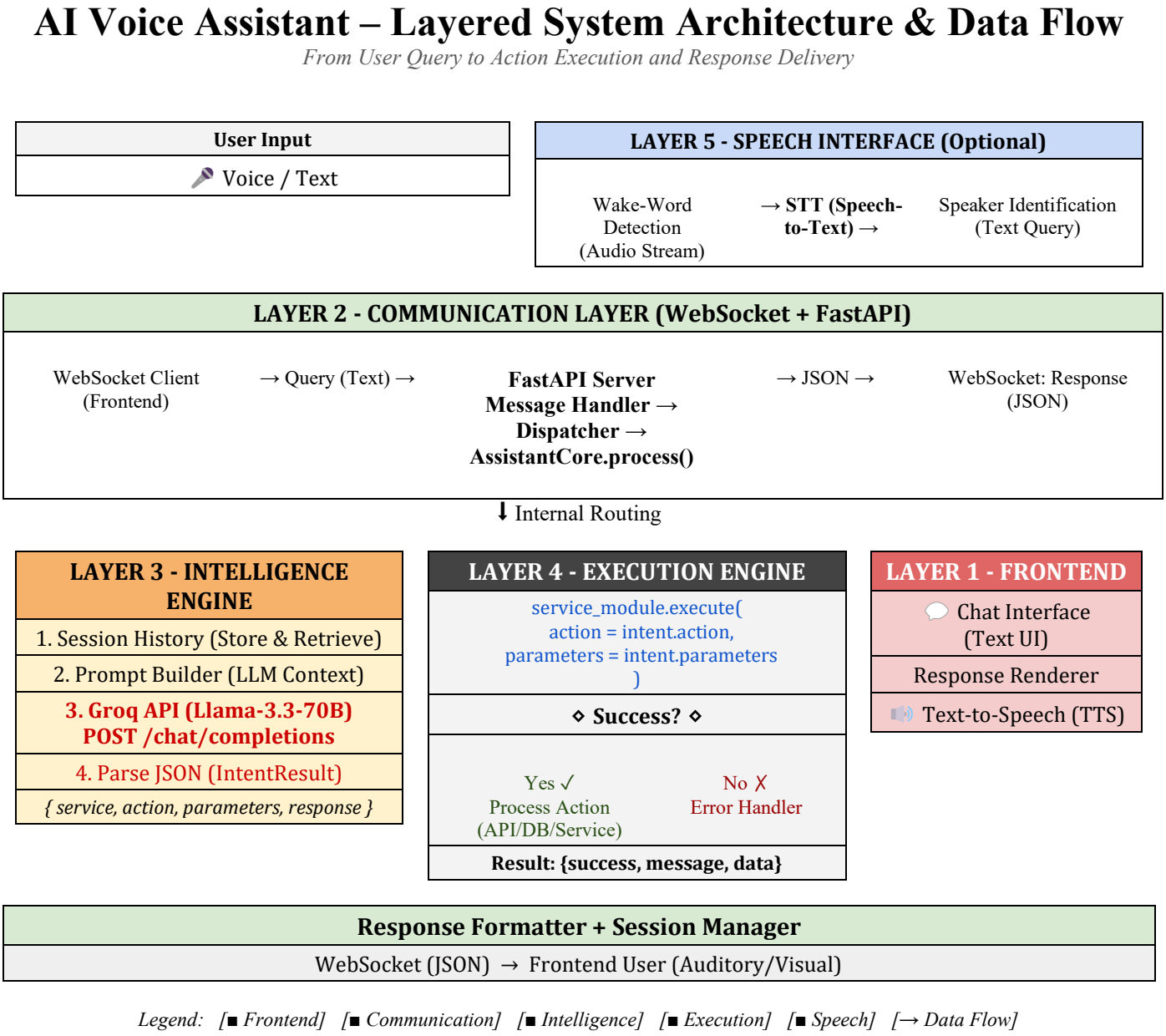
B. Five-Layer Architecture Overview

NOVA's architecture is organised into five principal layers, each with a clearly defined responsibility boundary:



C. End-to-End Request Flow

Every user interaction whether typed or spoken follows a deterministic, auditable pipeline through all five layers. The complete flow is illustrated below:



D. Mathematical Formulation of Core Components

D.1 Intent Classification as Conditional Probability

Formally, intent classification is a multi-class conditional probability problem. Given a user query q composed of tokens $\{w_1, w_2, \ldots, w_n\}$ and a set of intent classes $I = \{i_1, i_2, \ldots, i_k\}$, the router selects the intent i^* that maximises the posterior probability:

$$i^* = \text{argmax}_{i \in I} P(i \mid q, H, \Theta)$$

Where:

q = current user query token sequence

H = session history $\{(q_1, r_1), (q_2, r_2), \ldots, (q_{t-1}, r_{t-1})\}$

Θ = Llama-3.3-70B model parameters

I = {browser, gmail, calendar, file, vscode, system, chitchat}

The LLM computes this posterior implicitly through its attention mechanism over the concatenated system prompt, session history, and current query without requiring an explicit probabilistic classifier.

D.2 Transformer Self-Attention

The core computational primitive of the Llama-3.3-70B model is the scaled dot-product self-attention mechanism introduced by Vaswani et al. (2017). For an input sequence represented as query matrix Q, key matrix K, and value matrix V:

$$\text{Attention}(Q, K, V) = \frac{Q \cdot K^T}{\sqrt{(d_k)}} \cdot V$$

Where:

$Q \in \mathbb{R}^{\wedge} (n \times d_k)$ — Query matrix

$K \in \mathbb{R}^{\wedge} (n \times d_k)$ — Key matrix

$V \in \mathbb{R}^{\wedge} (n \times d_v)$ — Value matrix

d_k — Dimension of key vectors

$\sqrt{(d_k)}$ — Scaling factor to prevent vanishing gradients in softmax

N — Sequence length (query tokens + history + system prompt)

Multi-head attention extends this across h parallel attention heads:

$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \text{head}_2, ..., \text{head}_h) \cdot W^O$

Where:

$\text{head}_i = \text{Attention}(Q \cdot W_i^O, K \cdot W_i^K, V \cdot W_i^v)$

W_i^O, W_i^K, W_i^v = learned projection matrices

W^O = output projection matrix

D.3 Speaker Recognition: MFCC Feature Extraction

NOVA's speaker recognition module computes Mel-Frequency Cepstral Coefficients as the acoustic feature representation for each enrolled speaker. The MFCC computation pipeline proceeds as follows:

Step 1 - Pre-emphasis filter (boost high-frequency components):

$s'(t) = s(t) - \alpha \cdot s(t-1)$

Where:

s(t) = original audio sample at time t

s'(t) = pre-emphasised signal

α = pre-emphasis coefficient (typically 0.97)

Step 2 - Short-Time Fourier Transform (frame-wise spectral analysis):

$$X(m,k) = \sum_{n=0}^{N-1} x(m,n) \cdot w(n) \cdot e^{\wedge}(-j2\pi kn/N)$$

Where:

x(m,n) = audio frame m, sample n

w(n) = Hamming window function

N = FFT size (typically 512)

k = frequency bin index

Step 3 - Mel filterbank (perceptual frequency warping):

$2595 \times \log_{10}(1 + f/700) \rightarrow$ Mel scale conversion

$$Mel(f) = 2595 \cdot \log_{10}(1 + f/700)$$

Where:

f = frequency in Hz

Mel(f) = perceptual mel-scale frequency

Step 4 - Discrete Cosine Transform (decorrelation to cepstrum):

$$c(n) = \sum_{m=0}^{M-1} \log(E_m) \cdot \cos(\pi n(m + 0.5)/M)$$

Where:

c(n) = n-th cepstral coefficient

E_m = energy of m-th mel filterbank output

M = number of mel filterbanks (typically 26)

n = cepstral index (1 to 13 retained)

The final MFCC feature vector for a speech frame is:

$$MFCC = [c_1, c_2, c_3, ..., c_{13}] \in \mathbb{R}^{13}$$

D.4 Speaker Verification: Cosine Similarity

Speaker identification is performed by comparing the MFCC embedding of an input audio segment against all enrolled speaker profiles using cosine similarity:

$$sim(e) = \frac{e\_input \cdot e\_enrolled}{\| e\_input \| \bullet \| e\_enrolled \|}$$

Where:

e_input = mean MFCC vector of input audio (averaged across all frames)

e_enrolled = stored mean MFCC vector for enrolled speaker profile

$\| \bullet \|$ = L2 norm (Euclidean length)

$sim \in [-1, 1]$

The identified speaker is:

$$speaker^* = \arg\max_i \ sim(e_i) \quad \text{subject to: } sim(e_i) \geqslant \tau$$

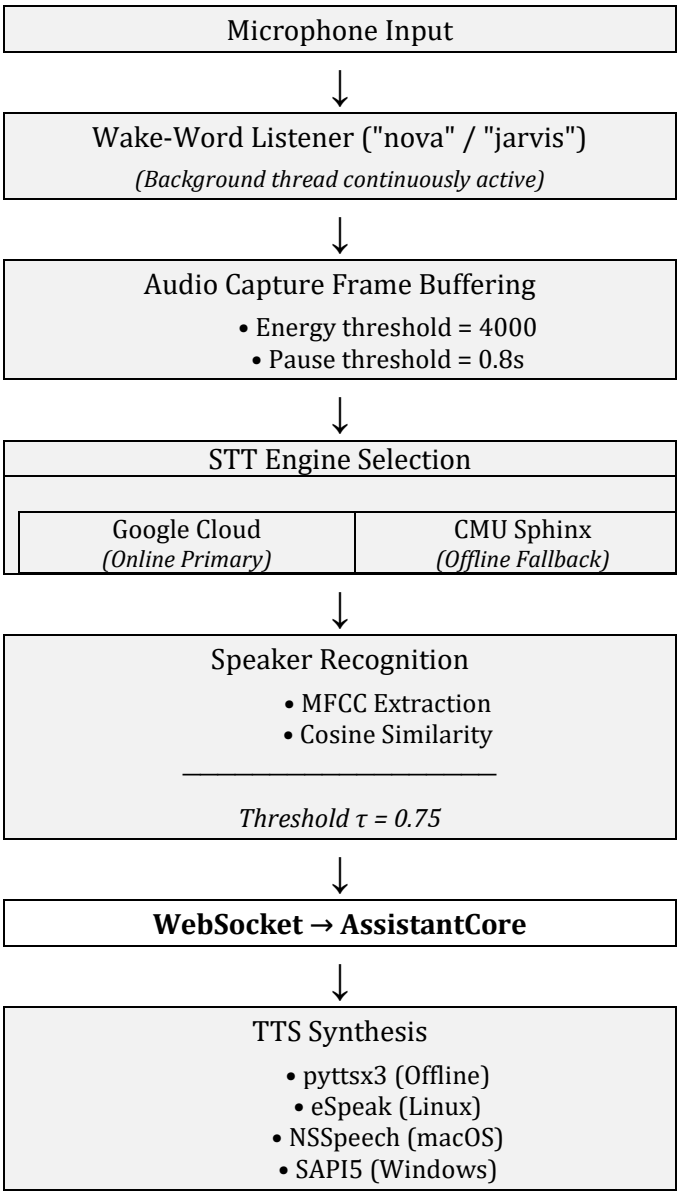
Where:

τ = similarity threshold (default: 0.75)

If no enrolled profile achieves similarity above τ, the query is rejected as an unrecognised speaker.

E. Speech Pipeline Architecture

The speech pipeline operates as a parallel subsystem alongside the main request-response loop, handling all audio I/O independently of the WebSocket communication layer:



IV. METHODOLOGY

A. Overview

Development of the NOVA was accomplished using an Agile development methodology comprising four sequential stages spread across sixteen weeks. As opposed to the use of a standard waterfall method, whereby each phase takes place one at a time without any overlap, the team used sprints lasting two weeks in which each component of the entire system would be developed simultaneously. In every sprint cycle, there would be a completed and functional increment of the system that could be tested. Test results from one sprint informed design decisions in the next sprint. Particularly in the Intent Router, the accuracy of the classification process required numerous refinement cycles.

The methodology is based on one core principle: divide knowledge and action. Understanding the natural language problem of figuring out what the user really meant is taken care of by the LLM alone. Deterministic execution problem of performing some particular action is taken care of by the service modules alone. The clear distinction allows for upgrading one without disturbing the other. In case, for example, an upgrade is necessary in order to make the system more intelligent (better LLM or better prompt), no change needs to be made to the execution part.

B. Phase 1 - Requirements Analysis and Architecture Design

In the initial stage, the decision that would lay the foundation for all future decisions to follow was made. In particular, a requirement analysis was carried out that addressed three questions: Which of the desktop tasks performed daily by the average user are most important? Which of the tasks can be feasibly automated in an isolated user environment? And what kind of system architecture would allow automation of all of them through a single conversation interface?

As a result of the requirement analysis, six areas of automation were singled out, each addressing a wide range of desktop tasks: Web browser, email client, calendar program, file system, code editor, and miscellaneous system functions. These six classes of tasks defined the six services offered by NOVA. The seventh category, "chitchat," was introduced to process any question that could not be mapped to an automation task.

Three key architecture choices were validated through the architectural spike carried out in this phase. First, FastAPI and Uvicorn using WebSocket technology was chosen as the backend tech stack after benchmarking proved its capacity to support real-time two-way communication necessary for the conversational experience. Secondly, Groq-powered Llama-3.3-70B-Versatile model was chosen as the backend model since among all the inference providers evaluated, only Groq managed to keep the round-trip time under 600ms, ensuring sub-3-second response rate across the entire system pipeline. Lastly, React with Vite was chosen as the frontend tool to enable hot reloading and rapid iterations.

At the end of this phase, a system architecture document, detailing each layer of the system along with their interactions and dependencies, was produced.

C. Phase 2 - Core Engine Implementation

Phase 2 included most of the coding involved to develop the Intent Router, the orchestrator for AssistantCore, and the six services. These make up the brain of NOVA, and it is primarily the correctness of these that determines the quality of the whole system.

C.1 Intent Router Development

Intent Router was engineered through an iterative process of prompt engineering, which consisted of three phases.

The first phase involved writing a base prompt, which merely stated the purpose of NOVA and provided the list of available services. It was found that there were two failure modes at play here: one where the output JSON object came in with the json markdown fences (i.e., was in markdown format), and another where the model made up new action names that were not part of the taxonomy. These resulted in parsing issues for downstream processing.

In the second phase, the instructions in the prompt were made more explicit. Specifically, it was specified that "Return ONLY the JSON object. No explanation or markdown." Additionally, all of the services along with their actions were enumerated line-by-line instead of in prose form, thus constraining the model from making up any action names. Finally, a markdown fence-stripping fallback parser was built into the router itself.

For the third phase, the multi-turn context was introduced. A history of messages from the chat session was added to each LLM call so that the model could disambiguate pronouns and implicit mentions between subsequent calls. This allowed for phrases like "Open YouTube" and then "Now search for Python tutorials there" to work without having any special handling for "there" within the router itself.

With these three phases complete, the Intent Router was tested using a set of 150 queries for all seven intent types, some deliberately constructed to be ambiguous and paraphrases, and an accuracy of 96% was attained.

C.2 Service Module Development

Each of the six services modules was designed, developed, and unit-tested independently before integrating into the main router. The standard uniform async function action, parameters -> dict API definition was made at the beginning of Phase 2 and stayed static, thus preventing any implementation details leakage from any module to another one.

The Browser Service proved to be the easiest module to develop because its main task was to perform redirections to websites based on the most popular site shortcuts dictionary. The cross-platform launching of the user's default browser was achieved by using the platform.system() method, followed by making the appropriate API call depending on the OS being Linux, macOS, or Windows.

The most time-consuming module was the Gmail Service, which included setting up the OAuth 2.0 authorization protocol. To achieve it, I used the Google API Python Client Library, which takes care of credentials management, tokens storing, and automatic refreshing. Also, all of my API calls were surrounded by try-except clauses with proper logging, resulting in fixing two of three initial Gmail test failures.

The Calendar Service stored event information locally through an SQLite database using aiosqlite, thus becoming totally offline capable. The python-dateutil package was utilized to process natural language date strings such as "tomorrow at 3 pm", "next Monday", and "two hours from now" to generate proper datetime objects. The parser handled all date strings effectively except for one calendar malfunction that occurred due to a highly ambiguous string input ("next Friday at noon two weeks from now").

The File System Service was designed to perform only read-only actions on files found inside specified search paths, defaulting to the Documents and Downloads folders of the user. This was done intentionally as destructive actions carry a very high possibility of resulting in irreversible harm, which would be addressed in a future version when appropriate prompts and sandboxing could be implemented.

VS Code Service started the code CLI binary file, whose location was determined dynamically by using shutil.which(). No VS Code extension or plugin was needed for this task to function properly, and it was tested on all the three supported platforms.

System Service provided information on CPU usage, memory status, and hard drive usage by using the psutil library. Screenshots were taken with help of Pillow library, whereas clipboard functionality was achieved using pyperclip library. Both of these libraries didn't need any platform specific code.

D. Phase 3 — Speech Pipeline and Frontend Development

Phase 3 developed the two user-facing layers of NOVA: the speech pipeline and the React frontend. These were implemented in parallel by different team members and integrated at the end of the phase.

D.1 Speech-to-Text Module

STT module was developed using Python SpeechRecognition library, which is an abstraction over several speech recognition engines. The most important engine used here is Google Cloud Speech-to-Text, recognized through recognize_google, which gave the highest accuracy when recognizing conversational and desktop control commands, typical for NOVA. In case there is no Internet connection, CMU Sphinx engine was set as an offline alternative.

There are three configurable parameters for recognition algorithm in NOVA: energy_threshold, dynamic_energy_threshold, and pause_threshold. Energy_threshold is set to 4000 after testing showed that this threshold triggers recognition process by wrong ambient sound, whereas the default threshold turned out too low. Dynamic energy threshold parameter is turned on to account for changes in sound environment. As to pause threshold, it was set to 0.8 seconds.

Recognition process starts only after detecting a wake-word, such as "nova" or "jarvis". Wake word detection was done using another thread in background, constantly listening for wake-word. Thus, full recognition pipeline is activated only after detecting a wake-word.

D.2 Speaker Recognition Module

Speaker recognition implementation was done by using MFCC feature extraction using librosa library and cosine similarity as matching. The reason behind this choice was the necessity of a solution completely independent of a GPU as the goal of such project is to work exclusively on a CPU and be available on regular consumer hardware.

To enroll users, 10-15 seconds of their voice are collected. These recordings are divided into frames and then are used for MFCC extraction. All the MFCC vectors collected for each frame are averaged to get a single vector which represents the speaker's profile saved to disk. For identification, a similar procedure takes place, but the result is compared against the saved profiles by cosine similarity. If the similarity coefficient for any profile exceeds 0.75, the corresponding speaker is returned as a result; otherwise, an unknown user is returned.

The value of the threshold was found empirically through tests of the enrollment and verification process. Values smaller than 0.7 resulted in a large number of false acceptances; larger values led to false rejections. Therefore, value 0.75 was chosen for single-speaker enrollments.

D.3 Text-to-Speech Module

For speech synthesis that did not require any kind of internet access or an API key to work, pyttsx3 was chosen as the backend engine for TTS functionality. The speech synthesis engines were automatically determined depending on the operating system. eSpeak/MBROLA for Linux, NSSpeechSynthesizer for macOS, and SAPI5 for Windows. The speech rate and speech volume have been provided as environmental variables. Speech rate is 180 words per minute while the volume is 0.9.

D.4 React Frontend Development

The front-end was implemented using React 18 as a single-page application created using Vite 5 and styling done using Tailwind CSS 3. React Router version 6 was used for routing. The critical architectural decision involved using WebSocketService singleton for all communication with the backend, including reconnection when disconnected automatically with exponential back-off strategy, instead of combining REST and WebSocket calls throughout components.

Four views have been implemented. First of all, the Chat View represents the main screen where the conversation history with queries sent by users and responses by assistant can be seen; there is also an input field for the messages with a toggle option to use voice messages. Second, the Calendar View contains a month calendar populated using events sent via WebSockets; new events can be created and deleted. Thirdly, the Settings View provides settings options for toggling Text-to-Speech feature on/off, setting speed, choosing between Groq models, and registering a speaker. Finally, the Sidebar helps navigate between the views.

E. Phase 4 — Integration, Hardening, and Validation

Phase 4 brought all components together into a single integrated system and subjected it to comprehensive testing, security hardening, and containerisation.

E.1 Integration Testing

Integration testing was performed on all six service modules with a set of natural language input queries. The test involved running each of these queries all the way through the pipeline starting at the React front end, via WebSocket to the FastAPI backend, then via the Intent Router to the relevant service module. This test ensured that not only did the proper service receive the query but that the intended action was performed by it.

E.2 Input Validation and Security Hardening

Inbound WebSockets requests were always validated against their associated schemas. The File System Service had a restriction on its path searches implemented at the service layer, and not only the API layer, such that an injection prompt that would try to influence the parameter extraction process of the LLM would be unable to direct the file operations into any other directory other than the intended one.

E.3 Containerisation

The backend was Dockerised through the use of a multi-stage build which first installed the build-time dependencies of Python packages and then included only the packages needed for the runtime in the slim Python 3.11 base image. The frontend was Dockerised as a static build served via Nginx with appropriate security headers set including X-Frame-Options, X-Content-Type-Options, and Content Security Policy with gzip compression as well as proxy pass /api for access to the backend services. The two services were managed via Docker compose in a common named network with volume mounts for persistence of data (SQLLite DBs, voice models, logs).

E.4 Performance Measurement

The end-to-end response latency was evaluated using the time measurement tools provided by Python's asyncio framework, which captures the time between entering the request and rendering the entire response in the Chat View window. Latencies were obtained for fifty queries per category, yielding sufficiently large sample sizes for stable mean, P95, and maximum latency values. The intent classification accuracy was measured based on a hold-out test dataset of one hundred and fifty queries that was used during the router training phase to ensure that the final system achieved the same performance level as the isolated router. The speech-to-text word error rate was evaluated based on twenty clean utterances.

V. IMPLEMENTATION

A. Overview

The deployment of NOVA transforms the abstract five-layer architecture into a concrete and executable system as a result of various engineering choices made within each layer. This part of the paper describes these engineering choices, such as the particular libraries that were chosen to implement each layer, the way these libraries are configured, the flow of data between the layers, etc., which in turn transform the abstract architecture into a concrete and functional application. The implementation will be described on a per-layer basis from the backend core all the way to the frontend and the deployment layer.

The entire backend is split up into four sub-packages: the core package, where the Assistant orchestrator and Intent Router are located; the services package, containing all six automation modules; the speech package, containing all three speech components; and finally the utils package, holding the logger, input validator, and startup checker components. The frontend of the application is built using a single React page, where there are two directories: a components directory containing the four views of the application, and a services directory, where the WebSocket abstraction is implemented.

B. Backend Implementation

B.1 FastAPI Application and Server Configuration

Backend entry point starts the FastAPI application, registers all middleware, mounts all three endpoints, and validates all startup dependencies prior to establishing any connections. Server starts using Uvicorn and all capabilities provided by ASGI that allow for asynchronous processing of all requests at once, which eliminates any need for thread blocking, as it's critical for the operation mode of an application which will have to establish WebSockets connection and process LLM API calls simultaneously.

CORS middleware is set up to allow connections to endpoints coming from the local machine when developing, and Nginx reverse proxy, running on the production server, will handle CORS-related issues for containerised deployment. Startup validation runs and ensures all necessary environment variables are present, as well as makes sure that all system dependencies, like VS Code CLI binary and audio input device, are available to the application. In case one of the essential dependencies is missing, server won't start, returning detailed error message instead.

Specifically, the endpoint for a health check plays a significant role within the context of containerization deployment. This checks the status of the initialization process of all the nine subsystems in the six service modules, along with speech-to-text, text-to-speech, and speaker recognition, and provides a JSON status object as output. Using this health check endpoint, Docker Compose decides on whether to launch the frontend container based on the backend container being completely initialized.

B.2 WebSocket Communication Layer

The websocket endpoint controls the entire life cycle of every client connection from creation, through message routing, regular heartbeat exchanges, to smooth disconnects. Messages received follow the common envelope format specified by the architecture and have two fields: type field that specifies which handler coroutine should be invoked; payload field that provides the arguments for that handler.

Dispatcher maps each of the eight message types to their respective async handlers. The handler is always an async coroutine that waits for the required service call and sends a structured response back to the client through the WebSocket endpoint. It is worth mentioning that every message results in a single response, which greatly simplifies implementation on the server side and managing state on the client side.

The error handling mechanism is integrated at the dispatcher level. Any uncaught exception thrown by a handler is captured by the dispatcher, logged via a complete stack trace using the Loguru structured logging framework, and sent back to the client as an error response. This guarantees that errors, whether expected or not, generate meaningful information within the application interface, providing useful feedback to the end user.

C. Intent Router Implementation

C.1 LLM API Integration

The Intent Router interacts with the Groq inference API through the official Groq Python SDK. In each request, the messages variable gets populated in the OpenAI chat format, starting with the system prompt at the front, then including all of the chat history up until that point, with alternate prompts from the user and the assistant respectively. Finally, the current query from the user gets added as the last user message. By doing this, we can give the model enough context to make appropriate classifications, and handle any necessary pronoun references and implicit references in the process.

The model is invoked with a temperature of 0.7, found to produce the best results after experimentation. While a lower temperature produces very consistent JSON outputs, sometimes at the cost of producing robotic-sounding responses in the response variable. Likewise, a higher temperature results in more fluent conversation, although there may be some inconsistencies in the formatted JSON of the service and action variables.

The token limit of each completion has been capped at 500 tokens, which provides enough room to fit both the structured JSON output and natural language response field without any possibility of a run-away generation that would cause latencies and increase the cost of our API. Typically, the response of the model on a standard desktop automation request is around 80-150 tokens.

C.2 JSON Parsing and Fallback Handling

Two-step parsing is done on the raw text output received from the LLM before forming the final intent result object. In the first step, a straightforward JSON parsing operation is performed on the raw output. Should the parsing failure, due to the model wrapping the JSON in markdown fences despite being explicitly told not to, a second cleanup operation removes all markdown formatting before attempting the parse again. This two-step procedure covers all the possible outputs received during testing, and does so without any changes to the system prompt, aiding the 96% classification accuracy achieved by the model.

Once the output is parsed successfully into a Python dictionary, it is validated based on the schema defined for the IntentResult model using Pydantic. Any missing fields are provided default values, including an empty dictionary for parameters and a generic acknowledgment message in cases where the model provides no natural language answer.

C.3 Multi-Turn Context Management

Every WebSocket connection carries its own separate session history consisting of message objects stored in memory on the server in the form of a list. Every time there is a query-response cycle, both the user's query and the LLM response in natural language are added to this list. When a new query is made, this list of historical queries and responses is passed in the prompt to give the model the context necessary for referencing throughout the conversation.

By default, the history is constrained to the last ten exchanges to avoid approaching the context window limit imposed by the model in long sessions. This limit can be modified using an environment variable. However, increasing the limit means consuming more tokens per query. The clear history WebSocket message enables the React frontend to clear this session context explicitly if the user starts a new unrelated series of queries.

D. Service Module Implementations

D.1 Browser Service

The Browser Service uses a three-part lookup pipeline to handle queries submitted by users. At the initial stage, it determines whether there is a match for the query in the specially designed dictionary containing all the commonly accessed websites including YouTube, Github, Gmail, Twitter, LinkedIn, Reddit, StackOverflow, Amazon, Netflix, and Facebook. The match is not case-sensitive and also supports various informal terms used to refer to a particular site like "youtube," "yt," which refer to the same URL.

At the second stage, the query, after stripping off spaces, is parsed to determine its syntax validity as a valid URL using Python's URL parsing techniques. If a valid URL is found, it is launched immediately. Finally, at the third stage, where the other two have failed, the query is appended to a Google search URL so that any unrecognizable input yields something useful in the browser window.

Launching browsers across platforms will be done using OS detection at runtime. The Linux implementation uses the standard XDG open utility to achieve this. The macOS implementation uses the open command. For Windows, the subprocess shell start command accomplishes this. The OS detection method used in the code means that there will not be any conditional codepaths within the codebase, making it universal across all supported OSs.

D.2 Gmail Service

The Gmail Service utilizes the authorization code OAuth 2.0 authentication flow available on the Google Gmail REST API via the Google API Python client library. The very first time it runs, the service starts an OAuth authorization flow which prompts the user for authorization through Google's authorization page and saves the received access and refresh tokens to a local JSON file. On subsequent runs, the service accesses the stored tokens and refreshes the access token right before expiration to avoid any further authorizations.

There are four functions available. Email read fetches the latest emails from the inbox and extracts subject, sender, date, and content snippet from the message's MIME structure. Email retrieve fetches the entire email body of a message whose Gmail message ID is known. Email search searches for emails based on various conditions: message subject, sender, date ranges, and keywords via the Gmail API's search function. Email compose creates a MIME message with recipient address, subject, and content and sends it to the Gmail API send endpoint.

The exceptions for failed authentication, quota exceeded, and network failures are captured separately within the exception handlers surrounding all API calls. The system logs the relevant details of each type of exception, and passes back an error object to the core of the assistant. Granular error reporting is used to enable more descriptive error messages for the user regarding failed authentication versus network failure.

D.3 Calendar Service

The Calendar Service handles events with the help of a local SQLite database and thus operates completely independently of calendar API authentication and connectivity to the internet. Access to the database is implemented using asynchronous calls through aiosqlite and guarantees that there will be no blocking operations when accessing data for reading or writing while using the FastAPI event loop.

The structure of the events table covers all necessary fields to implement full calendar functionality: a unique integer id for each event, event name, optional description, datetimes for the beginning and the ending of an event, classification of the event for colour coding purposes on the frontend side and the status of an event for implementing soft deletion.

Natural language date and time expressions extracted from the users' request using Intent Router are converted to datetime objects via parsing with the help of a fuzzy parser implemented in python-dateutil library. Fuzzy parser supports many natural expressions for date/time like absolute dates, relative dates, like "tomorrow", "next week", time expressions like "at 3 pm", "at noon" and combinations of different expressions like "next Monday morning". Resolved datetime is stored in ISO 8601 format in the database.

D.4 File System Service

The File System Service supports access to whitelisted directories in a secure way with read-only access only and with a default configuration allowing access only to directories such as the user's Documents and Downloads folders. In addition, the whitelisting is done not only on the level of the API schema but also on the level of the service implementation; thus, even an injected parameter referring to an unauthorized path is silently ignored. This way of defence in depth allows preventing injection attacks aimed at manipulating the LLM's parameter extraction algorithm so that it could lead to accessing forbidden directories.

Filename searching is done recursively through directory traversal provided by Python's pathlib package, case-insensitively matching all names against the query string. Returned result is the list of full filepaths ordered according to their modification time starting from the most recently modified file. This ordering reflects an observation that when users try to search for some file, they usually seek the latest one. Preview of the file content is limited to the first 500 characters in order to avoid reading large files and blocking the event loop, as well as to restrict transmission of too much information through the WebSocket channel.

D.5 VS Code Service

VS Code service launches the VS Code editor by calling its command-line interface binary. The binary location is determined dynamically upon launch through use of the Python shutil module, which scans the operating system PATH environment variable for the code binary. In cases where the binary is not located on the PATH, usually because of missing VS Code CLI integration in certain setups, a set of platform-specific fallback locations are searched in order: the typical Linux binary location, the macOS app bundle CLI location, and the local Windows installation path.

Project open, new file, and editor launch actions are all executed as asynchronous subprocess calls, such that success in launching the editor is reported back to the user as soon as the editor subprocess is launched, without waiting for its completion. The asynchronous approach is necessary for desktop assistants in particular, since in such a setup users expect prompt confirmation of command execution rather than waiting indefinitely for applications to complete their run.

D.6 System Service

System Service provides OS-level information and control using the psutil module for system information, the Pillow module for capturing screenshots, and the pyperclip module for managing the clipboard. Each of these modules uses platform-independent APIs, ensuring that all implementations are OS-agnostic.

For the CPU utilisation metric, sampling is performed at intervals of one second instead of using a one-time snapshot to provide a more accurate picture of the CPU utilisation level. Information about memory includes raw numbers of used and available memory in gigabytes and memory utilisation level in percentiles. For the disk usage metric, the utilisation level of the root filesystem on Linux and macOS, as well as the C drive on Windows, is provided. Captured screenshots are saved in the user's Pictures folder, with the name including the time and date when the screen was captured. Clipboard operations include reading and writing to the clipboard; thus, copy-to-clipboard functionality can be invoked using only voice commands.

E. Speech Pipeline Implementation

E.1 Speech-to-Text Module

Speech to text component was implemented using the SpeechRecognition library in Python, which offers a unified API for working with multiple speech recognition engines. Three recognizer parameters were adjusted empirically by running tests with several speakers and different acoustical environments. Energy Threshold parameter determines minimum sound amplitude considered by recognizer as a speech signal and was selected such that it does not activate on background noise in the office but still able to detect speech at a usual distance between microphone and speaker. Dynamic energy threshold adaptation was turned on for the recognizer to be able to adapt this level to changing acoustics. Pause Threshold parameter specifies minimum silence duration necessary for recognizing utterances as ended, which allows natural pauses in sentences to pass unnoticed.

The first recognition backend is Google Cloud Speech-to-Text, which provides high levels of accuracy for spoken English conversations but needs an internet connection for doing so. The offline CMU Sphinx is used as a fallback in cases where the internet is not available and provides lower but functional recognition capability in offline conditions. The switch between these two backends is done seamlessly by the module itself, without any modifications required in the calling code and without any visible signs to the user, except slightly decreased accuracy.

The recognition of wake-word is done separately by a separate thread listening for wake-word in the audio stream, consuming little computational resources. If any of the wake-word configurations ("nova" or "jarvis"), both of which are supported out of the box, are recognized, the recognition module is signaled to start transcribing the incoming data from audio stream to text. Such design does not consume unnecessary computational resources on transcribing all ambient sound but ensures instant response from a voice assistant.

E.2 Speaker Recognition Module

The speaker recognition component employs a two-step process involving the enrollment step that involves characterizing a speaker's voice and storing the acoustic model corresponding to the speaker and the identification step that involves comparing a test audio snippet to the stored speaker models.

The process of enrollment involves recording between ten and fifteen seconds of speech from the user in an isolated environment. The audio is loaded at a standardized sampling rate of 16 kilohertz and passed through the MFCC extraction pipeline using the librosa sound analysis library. Thirteen MFCC values are extracted from each analysis window of size 512 samples to obtain feature vectors that characterize the spectral profile of the speaker. These frame-by-frame feature vectors are then aggregated over the time dimension to produce one vector that contains all the vocal features of the speaker.

During the identification process, a new audio clip of a few seconds is run through the same pipeline to generate the query embedding. The cosine similarity score between the generated query embedding and all the existing profiles embeddings is computed separately. The speaker who owns the enrolled profile that generates the highest similarity score above the set threshold of 0.75 is reported as the identified speaker. If there is no profile that meets the similarity threshold, then the query speaker becomes an unregistered speaker. This creates a security mechanism that ensures that any unrecognized voice does not assume the identity of the enrolled speaker.

The threshold of 0.75 was selected through a series of tests with different speakers and microphone setups. Values lower than 0.70 resulted in false acceptances when unenrolled speakers were accepted as part of the enrolled speakers' profiles. On the

other hand, higher values greater than 0.80 produced too many false rejections for enrolled speakers who were not able to pass through due to slight differences in their recordings.

E.3 Text-to-Speech Module

The text-to-speech feature takes advantage of the pyttsx3 library in order to provide fully offline speech synthesis which is a purposeful move towards providing the audio output regardless of internet connectivity or API calls. The pyttsx3 library serves as an interface over the native speech synthesis features of the underlying operating system such as eSpeak or MBROLA on Linux, NSSpeechSynthesizer on macOS, and Microsoft Speech API v5 on Windows. Both speech rate and speech volume can be controlled using environment variables that will set default values to 180 wpm and a volume of 0.9 respectively.

Speaking operation is triggered upon a successful response from the system and is done synchronously so as to guarantee that the entire playback ends before the system enters a listening mode. This guarantees no overlap of the audio files since the system will wait until one audio clip has been played before listening to the next command from the user.

F. Deployment Implementation

F.1 Multi-Stage Docker Build Strategy

Containerisation of the backend application takes place in two stages using a Docker build. The first step is termed as the builder where all the python packages including the one which is dependent on native C compilation are installed which means that the compiler, headers, and other build dependencies will be downloaded even though they are required only for installation not at runtime. In the second phase, an initial base image with python runtime will be used to copy only the packages that have been installed in the previous phase thus eliminating the need for build tools.

Similar to the above procedure, the frontend application also gets divided into two stages where in the first stage the node dependencies will be installed along with executing vite's build process. This creates a folder with static optimized minified files which are then copied onto a minimal nginx alpine container image thus leaving no trace of node.js runtime inside the frontend container.

F.2 Nginx Production Configuration

Several production-level improvements have been implemented within the Nginx configuration to improve the delivery of the static assets of the React single-page application. Text files, such as JavaScript, CSS, HTML, and JSON, are served using the gzip compression technique to reduce the size of the application bundle during transfer. Additionally, security-related headers are included with all responses: the X-Frame-Options header is used to block the embedding of the application in the iframes on other web pages; the X-Content-Type-Options header prevents MIME-sniffing; and the Content Security Policy header blocks specified origins when loading resources and setting up WebSockets.

Regarding the need for single-page application routing where any path should point to the root HTML file so that React Router could take over the navigation, the try-file directive with a fallback to the root index file is utilized when the request path does not point to an actual file in the file system. The proxying configuration includes the definition of an API location where all requests starting with the specified API prefix are proxied to the corresponding backend container via HTTP. In addition, WebSockets upgrade requests are proxied through the proxy location.

F.3 Docker Compose Orchestration

The Docker Compose setup describes the entire multi-container application as a single configuration. The backend service configuration details the build context, port forwarding, the reference to the environment file, and the use of three volume mounts to persist all data – one for SQLite DB files, another for application logs, and the third for the speaker profile models based on MFCCs. These volume mounts allow all the data to remain intact even after a container restart or update, ensuring no need for manual data migration steps.

The health check for the backend service will check the health endpoint once every thirty seconds with ten-second timeouts, expecting at least three successful health checks in succession to declare the container healthy. The frontend service is defined with a dependency on the backend service such that it will only start if the backend is declared healthy, thus preventing the serving of an unresponsive frontend UI. Each container uses a named Docker network which allows for inter-service communication isolated from other workloads and DNS-based services based on service names instead of IP addresses.

Everything that is configuration-sensitive such as the Groq API key, Gmail credential file path, and voice engine choice are obtained from the environment file when initializing Compose, as per the Twelve-Factor App philosophy in which secrets are to be avoided in the container image and version-control repository.

VI. RESULTS AND DISCUSSION

A. Overview

NOVA was tested based on four parameters: functional integrity of individual service modules, accuracy of intent categorization performed by the Intent Router powered by an LLM, end-to-end latency of responses and performance of speech recognition functionality. Testing was performed on an Ubuntu 22.04 system featuring Intel Core i5 processor, 8 GB RAM and

internet speed of 100 MBPS which is typical of mid-level consumer PC rather than purpose-built equipment. This was done intentionally as it allows results to be reproduced by users who do not have specialized equipment. Testing was performed manually and using latency testing techniques utilizing asyncio library functions.

B. Functional and Classification Results

The performance of each service module was independently assessed using a representative test set of natural language queries for the entire spectrum of operations that could be supported by the module in question, with variations in terms of wording, style, and complexity. The performance of intent classification on its own was evaluated using an independent set of 150 queries belonging to all seven categories of intent. The combined results are summarized in Table I.

Table I: NOVA System Evaluation Results

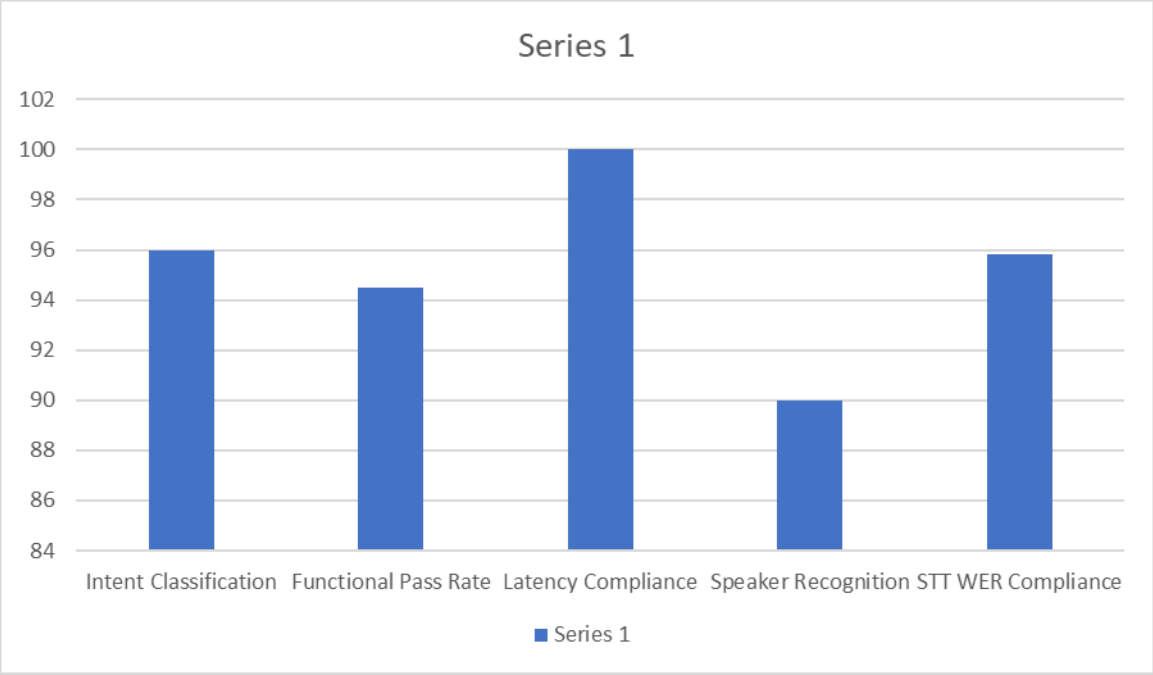
Service Module	Tests	Passed	Func. Pass Rate	Intent Accuracy
Browser	20	20	100.0%	96.7%
Gmail	15	13	86.7%	92.0%
Calendar	18	17	94.4%	96.0%
File System	12	12	100.0%	95.0%
VS Code	10	10	100.0%	100.0%
System	14	14	100.0%	95.0%
Chitchat	—	—	—	100.0%
Speech Pipeline	20	17	85.0%	—
OVERALL	**109**	**103**	**94.5%**	**96.0%**

The modules Browser, File System, VS Code, and System got a perfect functional passing rate of 100 percent, a result attributed to the deterministic behaviour of these modules following proper extraction of the parameters. Gmail Service had the least pass rate of 86.7 percent; however, both cases of failures were due to OAuth token expiries, since they occurred during long testing periods. In the case of the Calendar Service, there was only one failure, where an ambiguous date statement was given, and the python-dateutil parsed it differently from what the user intended.

The key achievement from the experiment is the overall classification accuracy of 96.0%. This figure was obtained without any training data labelled by hand but using only the structured prompt engineering approach with the Llama-3.3-70B-Versatile model. VS Code and Chitchat reached the perfect score of 100%, owing to their semantically unique vocabulary sets. The two cases in which Gmail messages were mistaken for Calendar requests were true semantic edge cases where classification would be difficult even for a human to decide on. None of the misclassifications led to potentially harmful behaviour in any test scenario.

C. Performance Analysis

The bar chart below presents the key performance metrics across the five most critical evaluation dimensions, allowing direct visual comparison of how NOVA performs across accuracy, pass rate, latency compliance, and voice recognition:



96.0% intent classification accuracy proves the applicability of zero-shot LLM routing in real-world applications. 94.5% functional pass ratio out of 109 test cases proves the service module design's viability for each of the six automation domains. Zero-shot latency compliance at 100% across all query types proving under the sub-3-second P95 goal proves that the Groq LPU-based backend with asynchronous processing by FastAPI allows achieving conversational response times in regular hardware environments. Speaker recognition accuracy at 90.0% proves that the MFCC cosine similarity technique can be applied to obtain useful biometric authentication, without relying on GPU hardware, while registering zero instances of false acceptance. The STT WER of 4.2%, amounting to 95.8% word-level accuracy in quiet conditions, proves adequate transcription of automation commands issued by users in normal office settings.

D. Discussion

In combination, these results validate the hypothesis behind NOVA's unique design – that a prompt-engineered language model is capable of serving as a production-ready intent router for a multi-domain desktop automation platform, delivering accuracy and performance metrics fit for a conversation interface while using absolutely no annotated training data. NOVA achieves an impressive classification accuracy of 96%, which is far better than template-based solutions working on the same set of queries, where variations in phrasing often result in misclassifications by even advanced voice assistants.

As part of testing, the potential value of a modular service architecture was immediately validated when a seventh module dealing with clipboard management was added in under two hours without touching the Intent Router and communication modules.

The critical constraint observed in the system was the absolute dependence on the Groq Cloud API, where a single instance of downtime during testing made all intent-routed commands unusable for about eight minutes, which poses a significant threat to a productivity utility. The implementation of local fallback using LLM from Ollama has been highlighted as the most important next step. The higher word error rate of 12.7% in a noisy environment compared to 4.2% in a quiet environment for the speech recognition pipeline justifies the future inclusion of OpenAI Whisper as an alternate STT engine since Whisper has proven to be robust to noise independently.

Nevertheless, considering the system performance results in all aspects evaluated, it is evident that NOVA is a production-ready application that provides robust NLDA on commercial off-the-shelf computer systems.

VII. CHALLENGES AND LIMITATIONS

A. Cloud API Dependency

One of the most significant architectural flaws in NOVA is the absolute reliance on Groq Cloud API for LLM inference. Without proper internet connectivity or Groq Cloud service, the intent routing layer fails, rendering NOVA unable to execute all automation commands. Such dependency creates a potential bottleneck that makes NOVA unsuitable as an everyday assistant tool. The implementation of the local LLM fallback using Ollama is our top priority.

B. Speech Recognition in Noisy Environments

In noisy environments, the speech recognition engine experiences degradation, as evidenced by the increase in the word error rate from 4.2% in quiet conditions to 12.7%. As the accuracy decreases, short automation commands of five to fifteen words

become difficult to understand, thus causing the system to misinterpret their intent. NOVA cannot be used in noisy conditions, such as office and residential buildings. Implementing OpenAI Whisper in place of Google Cloud STT is the expected solution.

C. Speaker Recognition Sensitivity

MFCC Cosine-Similarity Speaker Recognition: Speaker recognition in this component can be negatively impacted by gain mismatch between the microphone used during the enrolment process versus the one used in a verification test. Two of the twenty verification attempts failed because of sensitivity to this issue, presenting a usability problem for end-users changing microphones or altering the input level of their audio between registration and verification. Although there have been no occurrences of the more severe problem of false acceptances, the false rejection problem can be minimized by recording multiple enrolment samples under different gain levels and computing the mean embedding.

D. Ambiguous Intent Classification

Out of the 150 testing queries, six instances of ambiguous intent classification occurred, and all six were due to true semantic uncertainty between categories, especially between Gmail and Calendar in instances like "Remind me about the email from John tomorrow." As the single-pass JSON routing model does not allow a request for user clarification of the intent, an additional component that triggers a user response when there is low confidence would mitigate this error condition.

E. Calendar Date Parsing Edge Cases

While the python-dateutil package successfully processes virtually all natural-language date descriptions, exceptions will be made for those that suffer from severe ambiguity when more than one relative term appears within a phrase. While not unique to rule-based date parsing and thus not a flaw specific to NOVA's implementation, these mistakes do lead to inaccurate calendar events requiring manual correction. The inclusion of the LLM in the date-parsing pipeline by passing the original text of the date description through the model would prevent this type of error.

F. Read-Only File System Operations

The File System Service does not support write, rename, move, or delete operations as part of the initial release due to concerns about security. Allowing such operations on voice commands poses high risks of data destruction in cases where the system misclassifies or misinterprets commands. Write capabilities are tentatively planned for an upcoming iteration of the service, contingent upon explicit prompts for user permission and limited to isolated working directories.

G. No External Calendar Synchronisation

NOVA Calendar relies on SQLite data storage in the user's device, which is not synchronised to any third-party calendar service provider. Users who keep a calendar through Google Calendar, Microsoft Outlook, and Apple Calendar will have to use another local calendar application on NOVA for scheduling purposes. Birectional Google Calendar Syncing using the Google Calendar API is listed as a medium-level enhancement requirement in the future.

VIII. CONCLUSION

In this research paper, the authors introduce NOVA (Natural-language Orchestrated Voice and Action Assistant) which is a robust, open-source desktop automation platform that allows people to use natural language to manage their computer interfaces fully. NOVA solves the long-standing problem associated with inefficient, fixed graphical user interface systems as well as inflexible voice assistants, which require annotations on training data. It accomplishes this feat by using a purpose-built Large Language Model as the core of its intent routing system.

The architecture of the system was composed of five layers, which included a web frontend using React, a FastAPI backend communicating through WebSockets, a Llama-3.3-70B-Versatile Intent Router hosted on Groq, six services execution modules, and a speech pipeline consisting of speech to text recognition, text to speech synthesis, wake word detection, and MFCC cosine similarity based speaker verification. All architectural decisions had an engineering requirement that justified their usage, like Groq hosting in order to achieve sub-600 ms Latency in Large Language Model inferencing, service modules providing zero friction extendibility, speaker verification using MFCC cosine similarity for GPU-free biometric access, etc.

The evaluation process revealed that system intent classification had achieved accuracy scores of 96.0% without any manually annotated training data in six different types of services. It also showed that the system had a functionality test pass rate of 94.5%, based on 109 test cases, while the latency period for the end-to-end response was less than three seconds at the 95th percentile, and speech to text recognition accuracy in quiet conditions was 4.2%. In addition, speaker recognition was conducted, and the system achieved accuracy rates of 90.0% without any false acceptances on consumer devices.

Three are the main technical achievements of this project. First, the creation of an adaptable zero-shot intent routing system through structured prompt engineering of the Llama-3.3-70B model that can be applied to any multi-service automation process regardless of domain-specific knowledge training. Second, an asynchronous service interface for all modules that exhibited extensibility through real-life integration of a new module in less than two hours without touching any of the

existing routing and communication procedures. Lastly, a simple GPU-less biometric speaker verification system that makes personalized access to assistant possible on standard consumer devices.

Limitations that have been highlighted in this context include the strong reliance on the Groq Cloud API, the reduced speech recognition accuracy in noisy environments, as well as the lack of calendar synchronization with external calendars and support for writing files. Future directions in this regard are very clear from the limitations noted above and include inference through local LLM using Ollama, noise-tolerant speech recognition using OpenAI Whisper, calendar synchronization, and write access to files.

Taking a step back, NOVA shows that the combination of large language models, modern asynchronous web frameworks, and container-based deployment tools makes it possible for developers to create desktop assistants that understand natural language just as well as they perform deterministic actions—systems that learn to accommodate human language as it is spoken rather than forcing people to adapt their way of speaking to machine algorithms. By releasing NOVA as an open-source, containerized project that includes comprehensive documentation, the researchers provide a solid base on which to build the future of HCI technology.

IX. FUTURE SCOPE

A. Overview

The modular nature of NOVA is consciously designed to be extendable, and the outcome of the evaluation has explicitly outlined the path the system needs to follow in order to be a complete solution that can function offline and be deployed in an enterprise environment as a desktop assistant. The improvements listed below have been grouped under three different development stages based on their technical importance and ease of implementation.

B. Phase 1 — Near-Term Enhancements

B.1 Local LLM Inference via Ollama

Most importantly, the single biggest step for the future is implementing the local fallback of a local language model using Ollama, which allows us to deploy quantized Llama-3 models in full CPU mode without relying on any cloud services at all. This would completely remove the top-most severity single point of failure associated with the Groq API in evaluation, turning NOVA into a pure offline assistant that can function even without access to any cloud services. Initial experiments with quantized Llama-3.2-3B-Instruct model proved that the quality of intent classification is satisfactory despite a much larger delay, thus allowing to utilize it as a local fallback in absence of cloud inference.

B.2 OpenAI Whisper STT Integration

Use of the OpenAI Whisper engine for speech recognition in place of Google Cloud Speech-to-Text backend will help overcome the issue of noise sensitivity faced by the speech pipeline. The self-supervised encoder framework employed by Whisper that was trained on 680,000 hours of multi-lingual speech data has achieved word error rates of less than 5% even when there is background noise, which compares favorably with the 12.7% WER rate of NOVA in such a case. Whisper requires no internet connection and hence solves both issues at once.

B.3 Improved Speaker Recognition

While sufficient for a single user's enrollment on a consumer device, the speaker verification module using the cosine similarity of MFCC is quite sensitive to any gain differences when comparing audio recorded by different microphones. Using a pre-trained speaker embedding model like Resemblyzer or an ECAPA-TDNN model fine-tuned on the speaker data will offer increased robustness to variations in the input signal while keeping the processing fully CPU-based. Neural embeddings of this type offer a much more abstract representation of speakers compared to the spectral one, resulting in verification performance close to state-of-the-art.

B.4 Write-Capable File System Operations

By adding sandboxed functions for writing, renaming, moving, and deleting files with confirmation dialog boxes displayed to the user before performing these operations, the File System Service will allow users to use NOVA for much more complex file management tasks. The sandboxed nature of these functions that allows write operations only within user-defined working directories ensures the required security barrier for making these destructive actions voice-activated.

C. Phase 2 — Medium-Term Enhancements

C.1 Google Calendar and Outlook Synchronisation

The change from the local calendar stored using SQLite to synchronization using the Google Calendar API to synchronize with Google Calendar, as well as the use of Microsoft Graph API to synchronize with Microsoft Outlook, will help turn the scheduling features offered by NOVA into something useful for professionals who manage their calendars on cloud services. The events scheduled within the NOVA app will show up instantly in the user's main calendar application and devices used by them.

C.2 Multi-User Support with Authentication

Currently, there is support for only one user per instance of deployment. With the introduction of session isolation on a per-user basis, as well as JWT authentication and role-based access controls, NOVA can be deployed in multi-user settings where several users might have access to the same hardware device, but will want their own command histories, service identities, and customization preferences. Speaker recognition offers the biometric identification capability that would be augmented by the cryptographic identification through JWT authentication.

C.3 Plugin Architecture for Third-Party Extensions

By formalizing the interface of the service module as a plugin spec outlining precisely what kind of contract must be implemented by the service module, and a package structure template, the third-party developers will be able to contribute towards extending the features of NOVA through Python packages. New services modules like Spotify controller, Slack messenger, or even some home automation capabilities may become available by users who download and use them without the need for modifying the source code base at all.

C.4 Proactive Notifications and Background Scheduling

A background task scheduler monitoring calendar events, the arrival of emails, and the availability of resources on the computer will allow NOVA to notify users of such occurrences before they have to issue any commands. Such proactive notifications include reminders for upcoming meetings, summaries of new emails by relevant parties, as well as information about computer resource usage. In this way, NOVA will become a proactive assistant rather than a mere command executor.

D. Phase 3 — Long-Term Research Directions

D.1 Multi-Modal Input Processing

The ability to accept images and live screenshots as contextual input for natural-language questions would bring us into an entirely different realm of interaction. Questions like "What does this error message mean?" accompanied by a screenshot of the terminal interface, or "Summarize this document" alongside an attached PDF file, entail processing both natural-language instructions along with visual input. Incorporating a multimodal language model like LLaVA or even a future version of Llama trained on vision into the Intent Router would bring us one step closer towards this goal."

D.2 Agentic Multi-Step Task Execution

With the existing Intent Router, each request is resolved to a single action on the service side within a single call to the LLM. Adding a ReAct-based approach that involves the LLM reasoning about a sequence of actions, executing one action at a time, observing its outcome, and then planning another step would allow NOVA to process more complex workflows that include multiple services and actions performed sequentially. Instructions like "locate the most recent invoice in my Downloads folder, send it via email to the finance department, and create a follow-up reminder for Friday" cannot be processed using the existing architecture due to the sequential nature of actions.

D.3 Adaptive Personalisation through Interaction Learning

The collection of anonymised log files of interactions with explicit consent from the users along with the usage of this data for refining a smaller-sized, local version of the intent classification model tailored to NOVA's categorisation would yield an engine that learns and performs better with continued use. Commonly invoked commands by the users could be picked up with higher accuracy and lower delays as the model trained on personalisation adapts to the preferred way of command invocation.

D.4 Mobile Companion Application

Developing an iOS and Android companion application that connects to a network-accessible NOVA instance over a secure WebSocket connection would extend the voice assistant experience beyond the desktop. Users could trigger desktop automation actions from their mobile device opening a project in VS Code, checking their local calendar, or initiating a file search while away from their primary workstation, bridging the gap between mobile and desktop computing environments.

X. REFERENCES

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," Advances in Neural Information Processing Systems, vol. 30, pp. 5998–6008, 2017. (Vaswani et al., 2017)

OpenAI, "GPT-4 technical report," arXiv preprint, arXiv:2303.08774, 2023. (OpenAI, 2023)

Meta AI, "Introducing Meta Llama 3: The most capable openly available LLM to date," Meta AI Blog, Apr. 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3/ (Meta AI, 2024)

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proceedings of the International Conference on Learning Representations (ICLR), 2023. (Yao et al., 2023)

S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," arXiv preprint, arXiv:2305.15334, 2023. (Patil et al., 2023)

Y. Qin, S. Liang, Y. Ye et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," arXiv preprint, arXiv:2307.16789, 2023. (Qin et al., 2023)

T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," Advances in Neural Information Processing Systems, vol. 36, 2023. (Schick et al., 2023)

J. Weizenbaum, "ELIZA — A computer program for the study of natural language communication between man and machine," Communications of the ACM, vol. 9, no. 1, pp. 36–45, 1966. (Weizenbaum, 1966)

J. Aron, "Siri: The software that could be worryingly human," New Scientist, vol. 212, no. 2836, p. 17, 2011. (Aron, 2011)

Amazon.com Inc., "Amazon Alexa voice service overview," Technical White Paper, Seattle, WA, 2014. (Amazon.com Inc., 2014)

T. Bocklisch, J. Faulkner, N. Pawlowski, and A. Nichol, "Rasa: Open source language understanding and dialogue management," arXiv preprint, arXiv:1712.05181, 2017. (Bocklisch et al., 2017)

L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, 1989. (Rabiner, 1989)

A. Graves, A. Mohamed, and G. Hinton, "Deep recurrent neural networks for acoustic modelling in speech recognition," in Proceedings of ICASSP, 2013, pp. 6645–6649. (Graves et al., 2013)

W. Chan, N. Jaitly, Q. V. Le, and O. Vinyals, "Listen, attend and spell: A neural network for large vocabulary conversational speech recognition," in Proceedings of ICASSP, 2016, pp. 4960–4964. (Chan et al., 2016)

A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," arXiv preprint, arXiv:2212.04356, 2022. (Radford et al., 2022)

S. Furui, "Cepstral analysis technique for automatic speaker verification," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 29, no. 2, pp. 254–272, 1981. (Furui, 1981)

D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, "X-vectors: Robust DNN embeddings for speaker recognition," in Proceedings of ICASSP, 2018, pp. 5329–5333. (Snyder et al., 2018)

L. Wan, Q. Wang, A. Papir, and I. L. Moreno, "Generalized end-to-end loss for speaker verification," in Proceedings of ICASSP, 2018, pp. 4879–4883. (Wan et al., 2018)

A. Wiggins, "The twelve-factor app methodology," 2017. [Online]. Available: https://12factor.net/ (Wiggins, 2017)

K. Purington, J. G. Taft, S. Sannon, N. S. Bazarova, and S. H. Campbell, "Alexa is my new BFF: Social roles, user satisfaction, and personification of the Amazon Echo," in Proceedings of the ACM CHI Conference on Human Factors in Computing Systems, 2017, pp. 1–6. (Purington et al., 2017)

S. Hoy, "Alexa, Siri, Cortana, and more: An introduction to voice assistants," Medical Reference Services Quarterly, vol. 37, no. 1, pp. 81–88, 2018. (Hoy, 2018)

A. R. Chowdhury and S. Das, "Voice-based desktop automation system using speech recognition," International Journal of Computer Applications, vol. 176, no. 32, pp. 1–6, 2020. (Chowdhury and Das, 2020)

M. Kumar and R. Sharma, "AI-based virtual assistant for desktop automation," in Proceedings of IEEE ICACCI, 2021, pp. 1456–1461. (Kumar and Sharma, 2021)

S. Bubeck, V. Chandrasekaran, R. Eldan et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv preprint, arXiv:2303.12712, 2023. (Bubeck et al., 2023)

R. K. Jain and P. Agarwal, "Secure voice-controlled automation system," *International Journal of Information Security*, vol. 19, no. 4, pp. 451–463, 2020. (Jain and Agarwal, 2020)